



FILE PATH TRAVERSAL

TRAVERSAL SEQUENCES STRIPPED
WITH SUPERFLUOUS URL-DECODE

ACTIVIDAD 10

Índice

File Path Traversal: <i>Traversal Sequences stripped with superfluous URL-decode</i>	2
Especificaciones del laboratorio.....	2
Objetivo del laboratorio	2
Marco Teórico	3
Mecanismo del filtro y su bypass mediante URL-decode superfluo	3
Clasificación según OWASP Top 10 y triada CIA.....	3
Riesgos en un entorno real.....	4
Procedimiento	4
Explicación del Payload	9
Mitigación recomendada	9
Referencias.....	10

File Path Traversal: *Traversal Sequences stripped with superfluous URL-decode*

Especificaciones del laboratorio

Laboratorio: File Path Traversal, Traversal Sequences stripped with superfluous URL-decode

Nivel: Practitioner

Tipo de explotación: File Path Traversal, evasión de un filtro anti-traversal mediante doble codificación URL, explotando un URL-decode superfluo posterior a la validación

Objetivo del laboratorio

Explotar una vulnerabilidad de path traversal presente en la funcionalidad de visualización de imágenes de producto, con el fin de recuperar el contenido del archivo `/etc/passwd` del servidor. La aplicación bloquea el input que contiene secuencias de traversal explícitas y, adicionalmente, realiza una decodificación URL del valor recibido después de aplicar dicha validación. El objetivo específico es construir un payload que aproveche ese orden de operaciones para introducir una secuencia de traversal que solo se vuelve visible después del punto en el que el filtro ya dejó de revisar el input.

Marco Teórico

Mecanismo del filtro y su bypass mediante URL-decode superfluo

Esta variante del laboratorio implementa una defensa más estricta que las anteriores ya que bloquea directamente cualquier solicitud cuyo parámetro filename contenga la secuencia "../" de forma literal, sin importar si aparece una o varias veces. A diferencia del caso de saneamiento no recursivo, aquí el filtro sí detecta correctamente tanto la secuencia simple como las variantes anidadas, por lo que esos vectores no producen ningún resultado distinto de un 400 Bad Request.

El problema no está en la detección del patrón, sino en el momento en que ocurre. PortSwigger documenta este obstáculo como un caso en el que la aplicación primero valida el input recibido y después de aprobarlo, aplica una decodificación URL adicional sobre ese mismo valor antes de usarlo para construir la ruta del archivo (PortSwigger, s.f.). Esa segunda decodificación es "superflua" porque el servidor web ya había decodificado la solicitud una vez al recibirla, así que cualquier carácter que siga codificado en ese punto (por ejemplo, %252f) es indicio de que fue codificado dos veces intencionalmente por el atacante.

Al enviar el valor `..%252f..%252f..%252fetc/passwd`, la primera decodificación (la que hace el servidor web de forma transparente al recibir la solicitud) convierte %25 en el carácter %, dejando el string como `..%2f..%2f..%2fetc/passwd`. En ese momento no existe ningún "../" literal, así que el filtro anti-traversal lo deja pasar sin problema. Acto seguido, la aplicación ejecuta su propia decodificación URL adicional sobre ese valor ya aprobado, la cual sí interpreta %2f como el carácter /, produciendo finalmente `../../etc/passwd`, momento en el que la secuencia de traversal ya está fuera del alcance de cualquier validación previa.

Clasificación según OWASP Top 10 y triada CIA

Al igual que en los casos anteriores, esta vulnerabilidad corresponde a la categoría A01:2025 – Broken Access Control del OWASP Top 10 de 2025, la categoría de mayor incidencia registrada, presente en el 100% de las aplicaciones evaluadas (OWASP Foundation, 2025), y sigue mapeada a CWE-22 (MITRE Corporation, 2024). La diferencia frente a los casos previos no es la categoría, sino que aquí el filtro sí cubre correctamente el vector de entrada (detecta "../" en cualquiera de sus repeticiones), pero la falla aparece en una etapa posterior del procesamiento que el desarrollador no consideró parte de la superficie de validación: una transformación del propio dato ya validado.

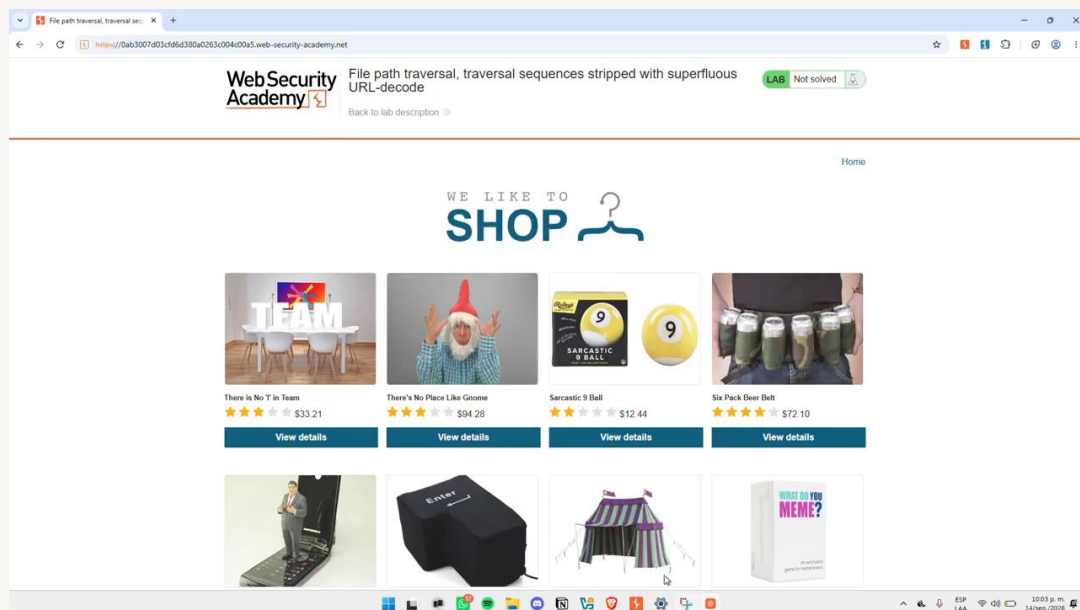
El impacto sobre la triada CIA sigue siendo el mismo que en los casos anteriores: comprometer la Confidencialidad al exponer archivos internos del servidor. Lo que distingue este caso es que ilustra un principio más amplio de validación de entradas, cualquier transformación o normalización que se aplique al dato después de validarlo puede reintroducir el patrón que la validación buscaba bloquear, por lo que la validación debe ejecutarse sobre la forma final y completamente resuelta del dato, nunca antes de una transformación pendiente.

Riesgos en un entorno real

Este patrón de vulnerabilidad, decodificar el input en más de una etapa del pipeline de procesamiento sin que todas las etapas de validación se ejecuten después de la última decodificación, es frecuente en aplicaciones que combinan un servidor web, un framework y código de aplicación propio, cada uno con su propio punto de decodificación URL implícito. El riesgo práctico es que un equipo de desarrollo puede confiar en que "el framework ya decodifica la URL", sin darse cuenta de que su propio código realiza una decodificación adicional más adelante en el flujo, dejando abierta una ventana de bypass que ninguna prueba basada en secuencias de traversal sin codificar lograría detectar.

Procedimiento

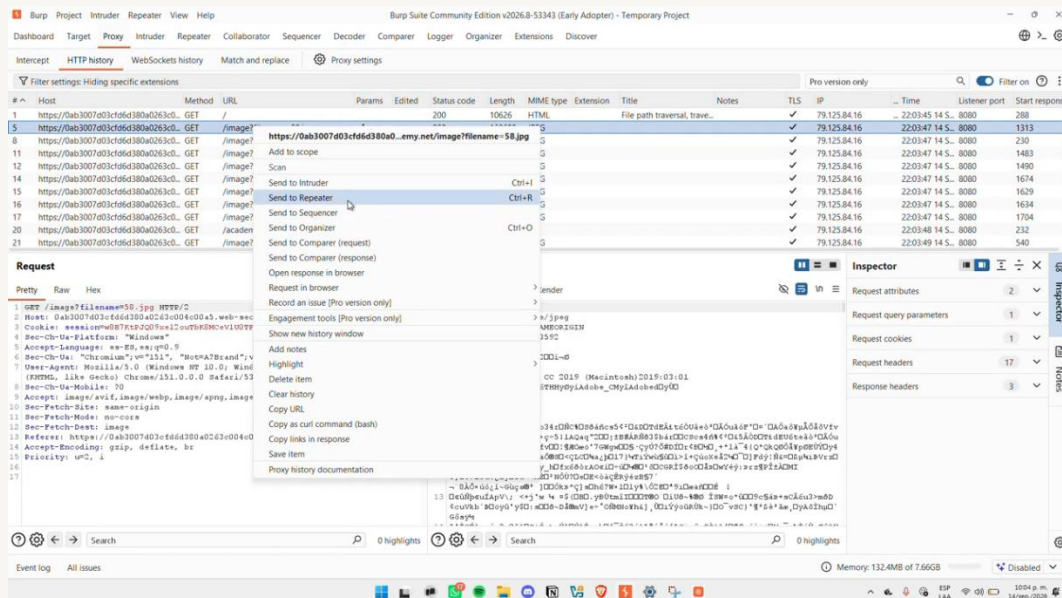
1. Se accede a la instancia del laboratorio "File path traversal, traversal sequences stripped with superfluous URL-decode" de PortSwigger Web Security Academy. El laboratorio inicia en estado "Not solved" y presenta una tienda en línea con productos cuyas imágenes se cargan dinámicamente.



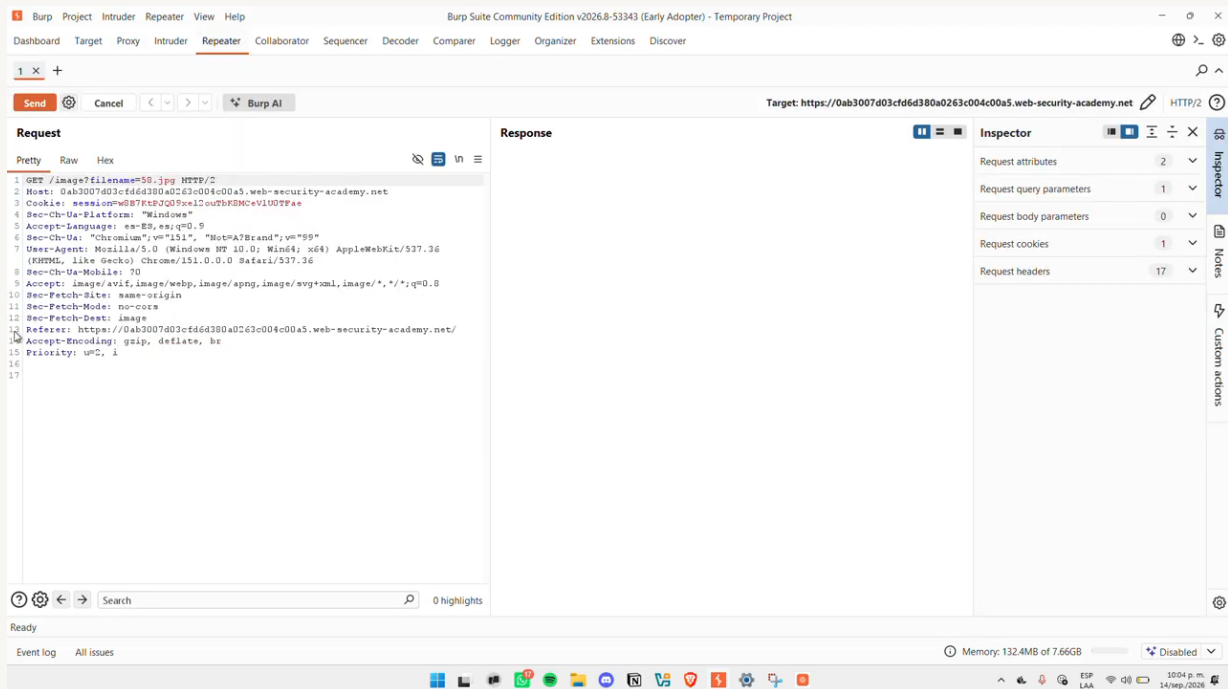
2. Con Burp Suite interceptando el tráfico, se navega por la tienda y se revisa el historial de proxy. Se confirma que cada imagen de producto se solicita mediante GET /image?filename=<archivo>.jpg.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type
1	https://0ab3007d03cfd6d380a0263c0...	GET	/			200	10626	HTML
5	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=58.jpg		✓	200	163688	JPEG
8	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=71.jpg		✓	200	179898	JPEG
11	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=40.jpg		✓	200	159107	JPEG
12	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=38.jpg		✓	200	277107	JPEG
14	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=74.jpg		✓	200	134940	JPEG
15	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=68.jpg		✓	200	83202	JPEG
16	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=47.jpg		✓	200	117709	JPEG
17	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=37.jpg		✓	200	111572	JPEG
20	https://0ab3007d03cfd6d380a0263c0...	GET	/academyLabHeader			101	147	
21	https://0ab3007d03cfd6d380a0263c0...	GET	/image?filename=6.png		✓	200	118082	JPEG

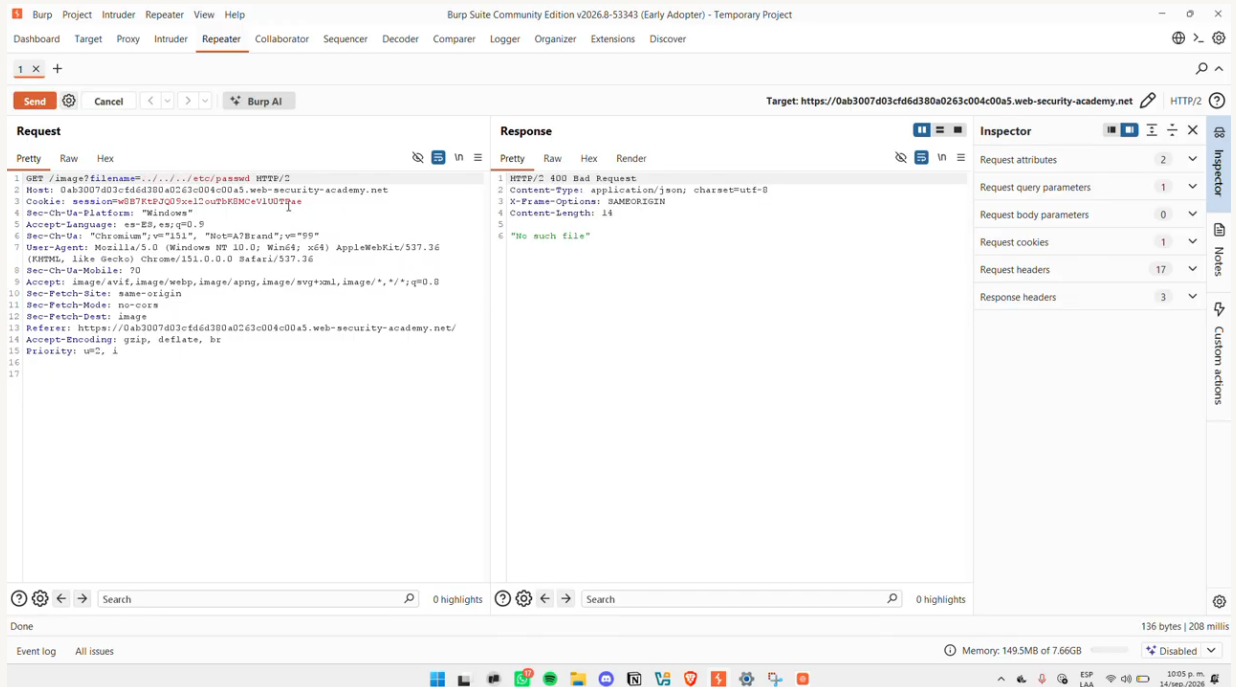
3. Se selecciona una de las solicitudes de imagen en el historial y, mediante el menú contextual, se envía al módulo Repeater para poder modificar y reenviar la solicitud de forma controlada.



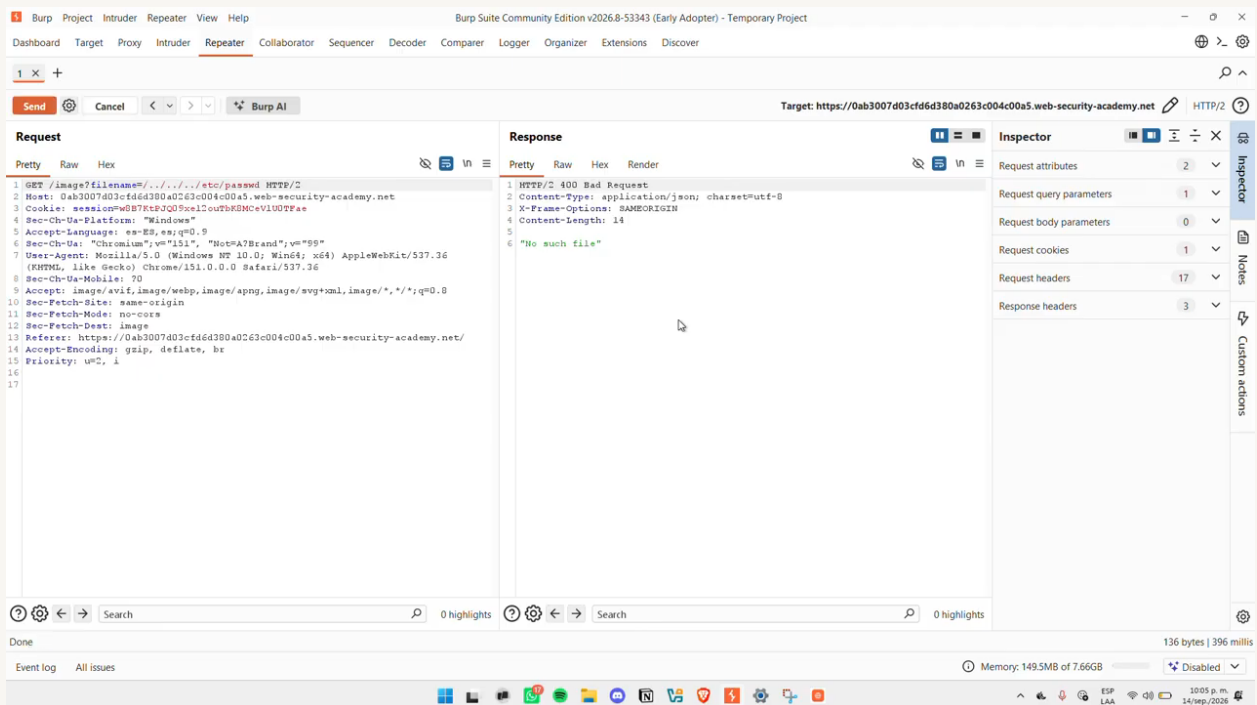
4. En Repeater se visualiza la solicitud original sin modificar: GET /image?filename=58.jpg, junto con las cabeceras enviadas por el navegador, incluyendo la cookie de sesión del laboratorio.



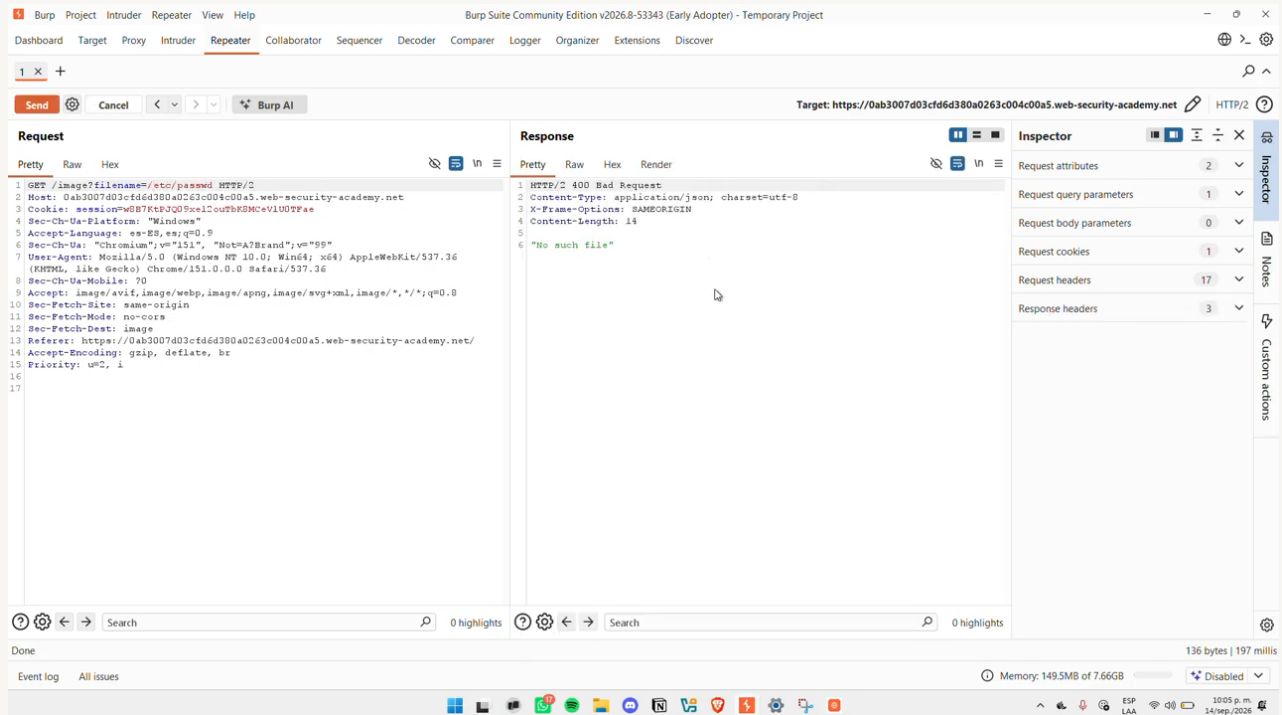
5. Se modifica filename con la secuencia ../../etc/passwd. El servidor responde 400 Bad Request con el mensaje "No such file", confirmando que el filtro detecta y bloquea la secuencia de traversal literal.



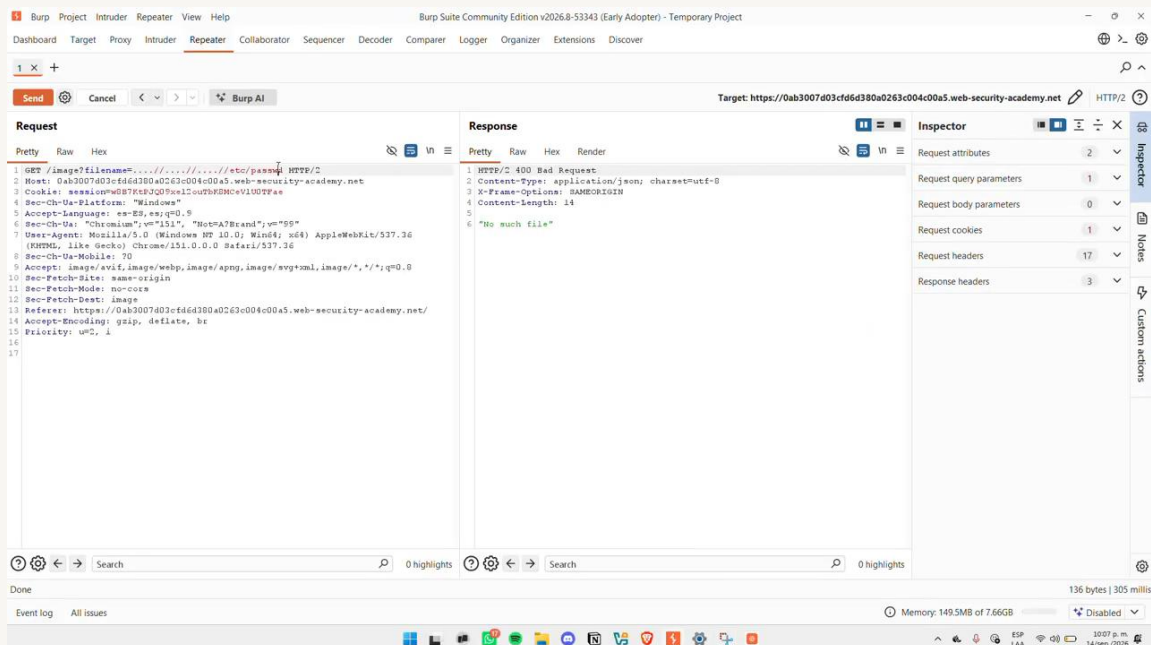
6. Se prueba `filename=/etc/passwd`, la ruta absoluta del archivo objetivo. La respuesta vuelve a ser 400 Bad Request ("No such file"), lo que descarta este vector para esta variante particular del laboratorio.



7. Se prueba `filename=../../../../etc/passwd`, la técnica de secuencias anidadas efectiva contra un saneamiento no recursivo. La respuesta es nuevamente 400 Bad Request, lo que confirma que, a diferencia de la Actividad 09, este filtro sí elimina correctamente las secuencias de traversal aunque estén anidadas.

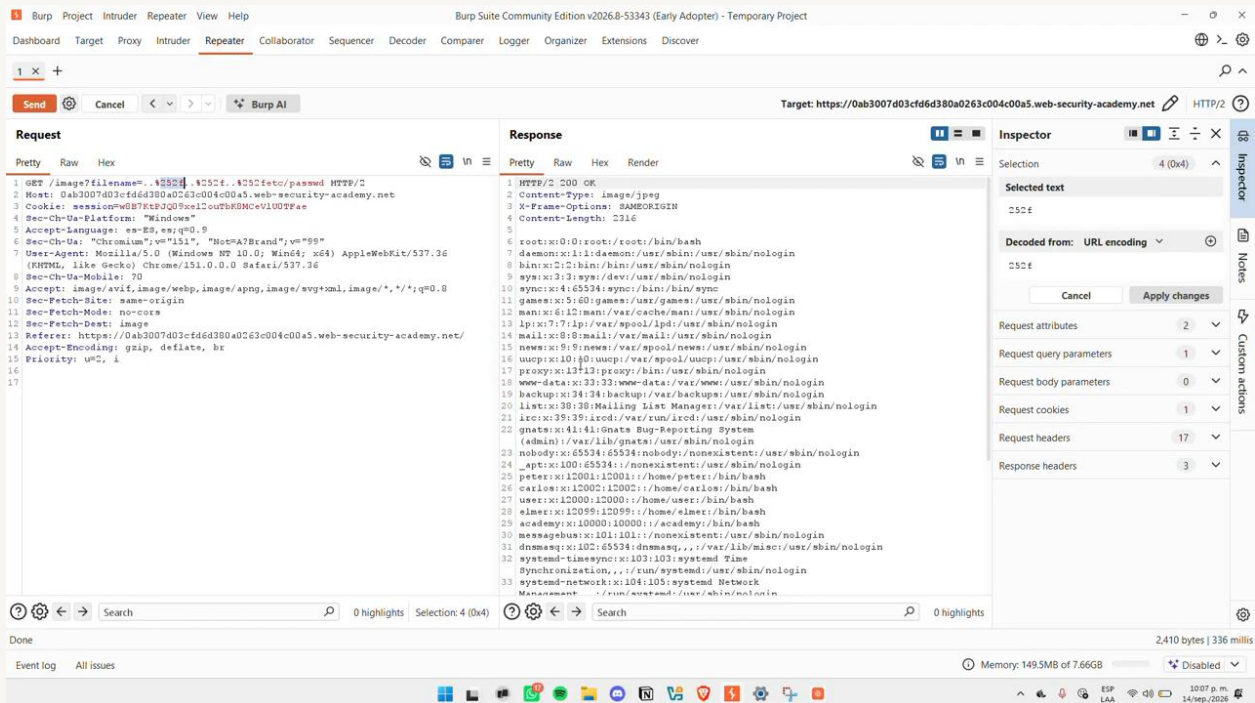


8. Se modifica filename con el valor `..%25f..%25f..%25fetc/passwd`, donde %25 es la codificación URL del carácter %. Este payload está pensado para sobrevivir a la validación del filtro y sólo revelar la secuencia de traversal después de una decodificación adicional.

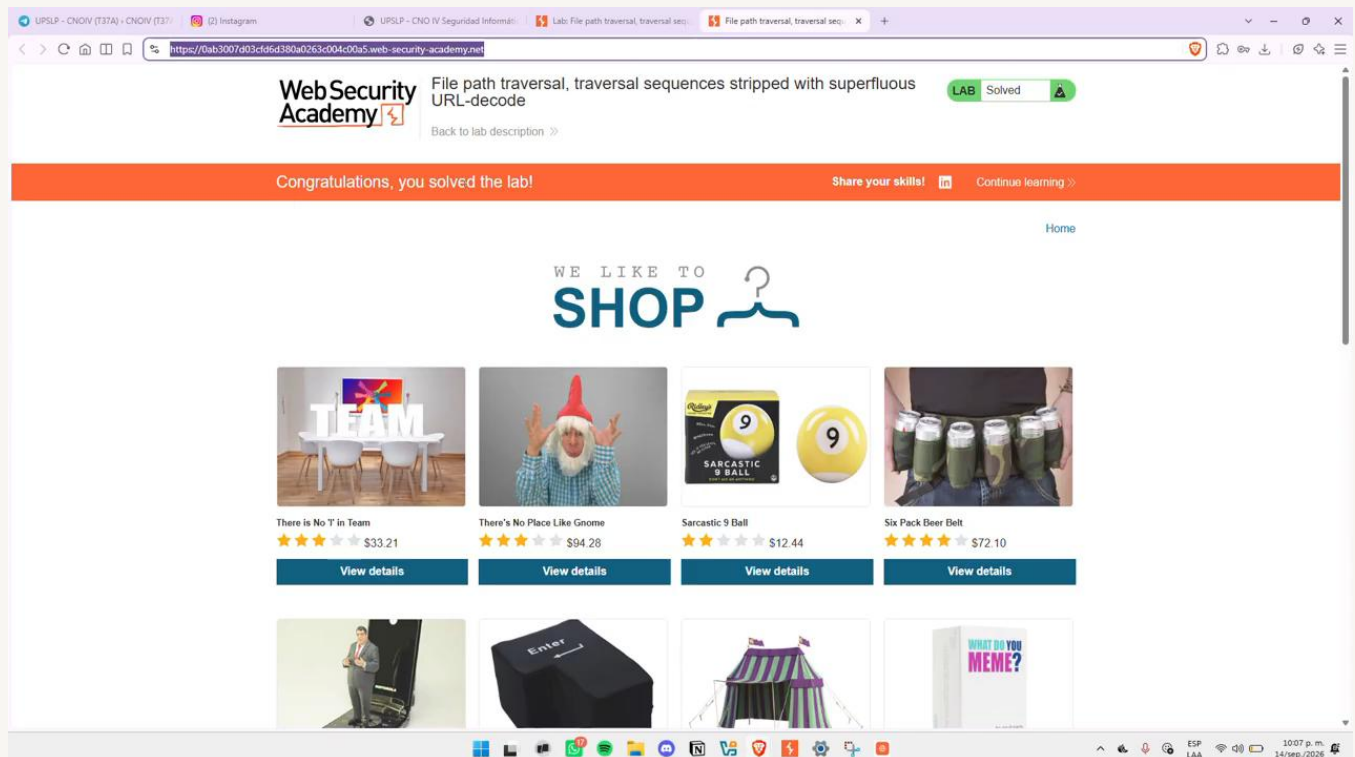


9. Al enviar la solicitud, el servidor responde 200 OK con el contenido completo del archivo `/etc/passwd`. El panel Inspector de Burp Suite, usado sobre el fragmento %25f de la solicitud, confirma que ese valor

corresponde a una codificación URL que, al decodificarse, produce el carácter / utilizado para reconstruir la secuencia de traversal.



10. Al regresar a la aplicación, el laboratorio muestra el estado "Solved" junto con el mensaje "Congratulations, you solved the lab!", confirmando que la explotación fue exitosa.



Explicación del Payload

El Payload que utilicé fue `..%252f..%252f..%252fetc/passwd`, como valor del parámetro filename. Este payload es distinto a los dos anteriores (ruta absoluta y secuencias anidadas), que ya fueron descartados. En vez de reordenar o repetir caracteres para ocultar el `"../"`, aquí se codifica dos veces, de modo que en el momento en que el filtro revisa el input, el patrón todavía no existe en esa forma.

Su efectividad depende de que la aplicación decodifique la URL en dos momentos distintos. Primero, el servidor web decodifica la solicitud una sola vez de forma automática al recibirla: `%25` se convierte en `%`, así que `%252f` pasa a ser `%2f`. En ese punto el filtro anti-traversal revisa el string, no encuentra ningún `"../"` literal (`%2f` todavía no es una barra) y deja pasar el valor. Después de esa validación, la propia aplicación ejecuta una segunda decodificación URL sobre ese mismo dato ya aprobado, y esta sí convierte `%2f` en `/`, reconstruyendo en ese momento la secuencia `../././` que finalmente se usa para localizar el archivo.

El filtro no tiene ningún defecto al detectar el patrón `"../"` en sí, como se confirmó con los pasos seis y siete, lo bloquea correctamente incluso en variantes repetidas o anidadas. Su punto débil es el orden de las operaciones, valida el dato antes de que termine de tomar su forma final, cuando debería hacerlo después de esa segunda decodificación.

Mitigación recomendada

- **Validar el input después de la última transformación y no antes:** cualquier decodificación, conversión de caracteres especiales a su forma literal o normalización que la aplicación vaya a aplicar al dato debe ejecutarse primero, y la validación anti-traversal debe correr sobre el resultado final que realmente se usará para construir la ruta.
- **Eliminar decodificaciones redundantes:** si el servidor web o el framework ya decodifican la URL automáticamente, sin que el desarrollador lo programe explícitamente, el código de la aplicación no debería volver a decodificar el mismo valor ya que cada decodificación adicional es una oportunidad para que un input doblemente codificado revele un patrón que ya debería haber sido bloqueado.
- **Validación estricta por whitelist, no por bloqueo de patrones (blacklist):** en lugar de buscar y rechazar secuencias específicas como `"../"` en cualquiera de sus codificaciones, aceptar únicamente caracteres alfanuméricos y una extensión de archivo predefinida, rechazando cualquier valor que contenga `"/"`, `"\"`, `"%"` o que no cumpla ese formato exacto.
- **Canonicalización y verificación de ruta base:** resolver la ruta absoluta canónica final mediante funciones nativas del lenguaje (`getCanonicalPath()` en Java, `os.path.realpath()` / `Path.resolve()` en Python) y verificar que el resultado se ubique estrictamente dentro del directorio autorizado, sin depender de que ninguna etapa de saneamiento previa haya sido correcta.
- **Principio de menor privilegio:** ejecutar el proceso del servidor con permisos mínimos, de forma que, aunque un payload doblemente codificado lograra evadir la validación de la aplicación, el sistema operativo continúe negando el acceso a directorios sensibles como `/etc` o `/root`.

- **Auditoría y pruebas específicas de bypass:** al probar defensas anti-traversal, incluir explícitamente payloads con codificación URL simple y doble, además de rutas absolutas y secuencias anidadas, en lugar de validar la corrección del filtro únicamente contra el vector de ataque original que motivó su implementación.

Referencias

MITRE Corporation. (2024). *CWE-22: Improper limitation of a pathname to a restricted directory ('Path Traversal')*. Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>

OWASP Foundation. (2025). *A01:2025 – Broken Access Control*. OWASP Top 10:2025. https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

PortSwigger. (s. f.). Lab: File path traversal, traversal sequences stripped with superfluous URL-decode. Web Security Academy. <https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode>

PortSwigger. (s.f.). *What is path traversal, and how to prevent it?* Web Security Academy. <https://portswigger.net/web-security/file-path-traversal>